

Artificial Intelligence

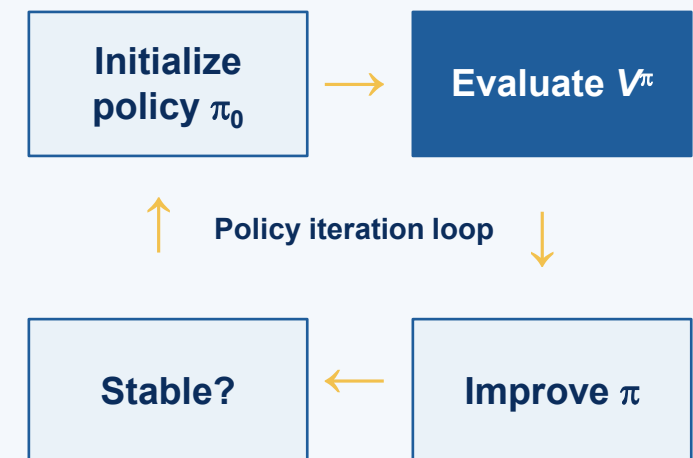
Lesson 14: Reinforcement Learning II

Policy Iteration

LOCK the arrows → update the numbers.
UNLOCK the arrows → choose better actions.

→ -1.9	→ 9.6	→ 16.1	+10
↓ -1.9	WALL	↓ -3.8	-10
→ -1.9	→ -1.9	→ -12.4	↑ -16.9

Same Gridworld. Different strategy.



Today's mission

Policy and value give meaning to behavior over time.

1 Explain policy evaluation and policy improvement.

2 Apply a simple policy-iteration step.

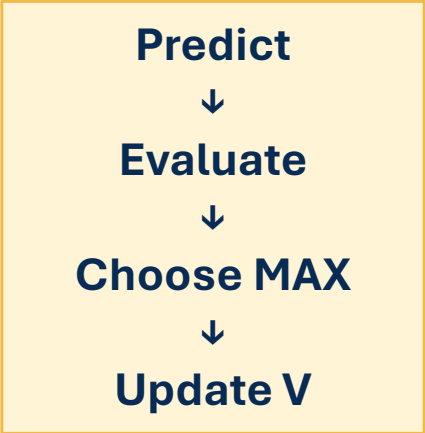
3 Compare policy iteration and value iteration.

Performance target: given a current policy and evaluated action values, determine whether the policy should change and explain when policy iteration terminates.

Retrieval: what did value iteration do?

Value iteration repeatedly applied optimality backups.

-1.9	9.6	16.1	+10
-1.9	WALL	-3.8	-10
-1.9	-1.9	-12.4	-16.9



$$V_{k+1}(s) = \max_a [R(s,a) + \gamma \sum P(s' | s,a) V_k(s')]$$

Think back

Why did the Bellman backup use MAX?

What did convergence look like?

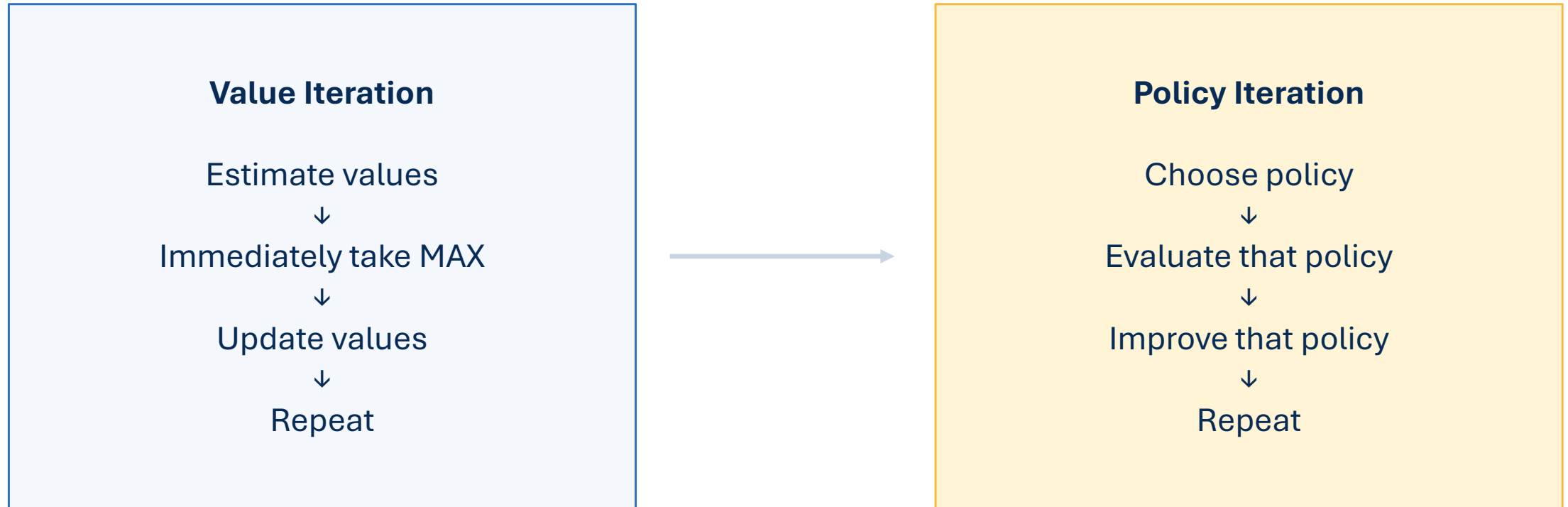
Today

We solve the same MDP, but organize the work differently.

Value iteration mixes judging and choosing inside each backup.

A different strategy for the same MDP

Two methods use the same model, but structure the computation differently.



Same states, actions, transitions, rewards — different organization of the reasoning.

Start with a policy — not with optimal values

Policy iteration begins with behavior that may or may not be good.

→	→	→	+10
↓	WALL	↓	-10
→	→	→	↑

Question
Is this policy optimal?

Answer
We do not know yet.
First we evaluate it.

The arrows specify the current policy.

Policy iteration starts by asking: “How good is the behavior I already have?”

The arrows are the current policy

A policy maps each state to an action.

→	→	→	+10
↓	WALL	↓	-10
→	→	→	↑

$$\pi(s) = a$$

Policy

- A rule for what to do in each state
- Not a single action (“East” is one choice)
- Tells us what to do whenever each state is reached

If the world is uncertain, the policy still tells the agent what to do after the outcome occurs.

Policy evaluation asks one question

How good is this behavior if we keep following it?

→ -1.9	→ 9.6	→ 16.1	+10
↓ -1.9	WALL	↓ -3.8	-10
→ -1.9	→ -1.9	→ -12.4	↑ -16.9

$$V^{\pi}(s)$$

Meaning

Expected cumulative discounted return starting in state s and following policy π .

Important

We are not yet searching for a better action.

**Evaluation judges the current arrows.
It does not change the arrows.**

LOCK THE POLICY

During policy evaluation, the arrows do not move.

→ 0.0	→ 0.0	→ 0.0	+10
↓ 0.0	WALL	↓ 0.0	-10
→ 0.0	→ 0.0	→ 0.0	↑ 0.0

**POLICY LOCKED
DURING EVALUATION**

What can change?

The value estimates in the cells.

What cannot change yet?

The action arrows selected by π .

Evaluation changes the numbers — not the behavior.

Tiny math retrieval

Same expected-value operation as L13; a different action-selection rule.

Next state	Probability	Value
X	0.80	5
Y	0.20	1

$$0.8(5) + 0.2(1) = 4.2$$

Today's difference

We only evaluate the action specified by the current policy.

Same arithmetic. Different question: “How good is this policy?”

Value iteration versus policy evaluation

The MAX moves to a different phase.

Value iteration

Evaluate all actions

$$V(s) \leftarrow \max_a Q(s, a)$$

Policy evaluation

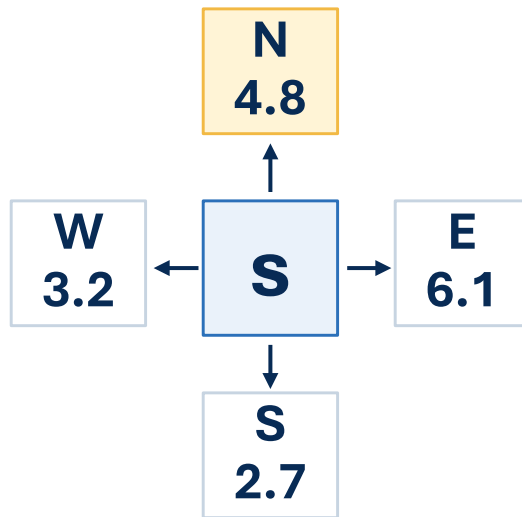
Evaluate only the policy action

$$V^\pi(s) \leftarrow Q^\pi(s, \pi(s))$$

Evaluation asks “How good is what I currently do?” — not “What should I do instead?”

Why there is no MAX here

A deterministic policy has already selected the action to evaluate.



current: N

During evaluation

Only the current policy action is active.
If $\pi(s)=\text{North}$, evaluate North.

Later, during improvement

We will compare action values and may
change the arrow.

The other actions are alternatives — but they are not selected during fixed-policy evaluation.

Evaluation backup: one state

Use the policy action and current successor values.

→ -1.0	→ -1.0	→ 15.0	+10
↓ -1.0	WALL	↓ -2.8	-10
→ -1.0	→ -1.0	→ -1.0	↑ -15.4

Policy action

$\pi(s) = \text{East}$

Outcomes

- 0.8 → intended cell
- 0.1 → slip north/stay
- 0.1 → slip south

$$V^\pi(s) = \sum P(s'|\pi(s)) [R + \gamma V^\pi(s')]$$

Key phrase
under the current policy

We are estimating what happens if the agent keeps behaving according to π .

Compute → Interpret → Predict

Every policy-evaluation number needs meaning.

$$V^{\pi}(s) = 5.0$$

Compute

The backup returned a value estimate of 5.0 for this state.

Interpret

5.0 is not an immediate reward. It is expected long-term return under π .

Predict

If successor-state values increase next sweep, this estimate may increase too.

Policy evaluation is about the value of continuing to follow this behavior.

One backup is not policy evaluation

Policy evaluation repeats backups until values stabilize under the fixed policy.

One evaluation backup

Update one state's estimate using the current policy action.

Policy evaluation

Repeatedly update values for the policy until the estimates are stable enough.

The policy stays fixed while value information propagates through the grid.

Evaluation Sweep 0

Initial estimates under the locked policy.

→ 0.0	→ 0.0	→ 0.0	+10
↓ 0.0	WALL	↓ 0.0	-10
→ 0.0	→ 0.0	→ 0.0	↑ 0.0

POLICY LOCKED

At the start, zero means initial estimate — not true value.

Evaluation Sweep 1

Values change. Arrows stay fixed.

→ -1.0	→ -1.0	→ 15.0	+10
↓ -1.0	WALL	↓ -2.8	-10
→ -1.0	→ -1.0	→ -1.0	↑ -15.4

POLICY LOCKED

The current policy begins to reveal which states lead toward good or bad outcomes.

Evaluation Sweep 2

More long-term consequences are reflected in the numbers.

→ -1.9	→ 9.6	→ 16.1	+10
↓ -1.9	WALL	↓ -3.8	-10
→ -1.9	→ -1.9	→ -12.4	↑ -16.9

POLICY LOCKED

Nothing about the behavior changed; only the estimates became more informed.

Why multiple sweeps?

Values depend on successor values, which may still be changing.



Sweep pattern

C learns about the terminal reward.
Then B can use C.
Then A can use B.

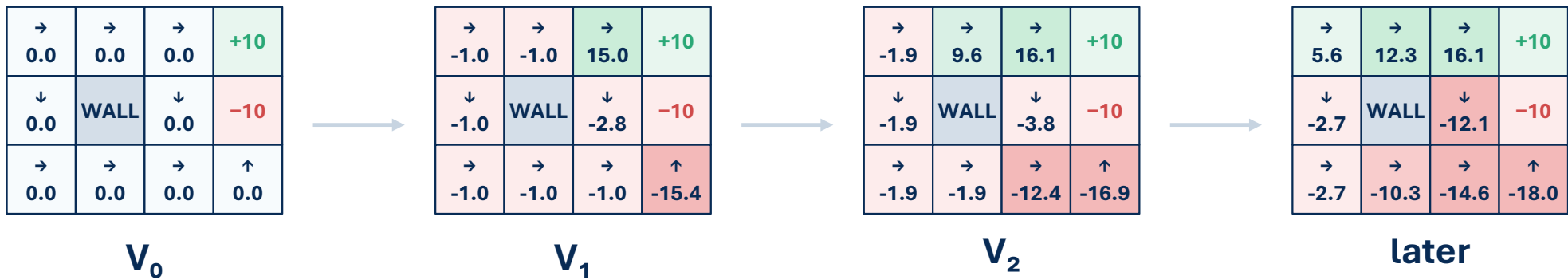
Key idea

Even fixed behavior can require repeated value backups.

Evaluation propagates long-term consequences through the locked policy.

Evaluation changes numbers — not behavior

Same arrows, evolving value estimates.



The action arrows stay locked throughout evaluation.

Now unlock the policy

Once values are stable enough, we ask whether to change behavior.

→ 5.6	→ 12.3	→ 16.1	+10
↓ -2.7	WALL	↓ -12.1	-10
→ -2.7	→ -10.3	→ -14.6	↑ -18.0

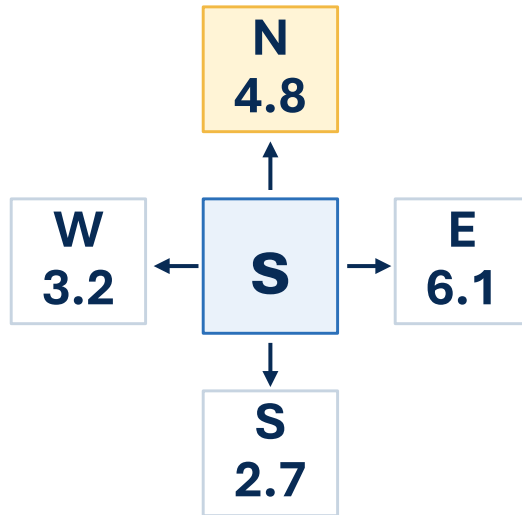
UNLOCK
FOR IMPROVEMENT

New question
Is another action better in any state?

Evaluation gave us numbers. Improvement uses them to reconsider the arrows.

Policy improvement asks: can I do better here?

Now we compare action values at a state.



current: N

Improvement phase

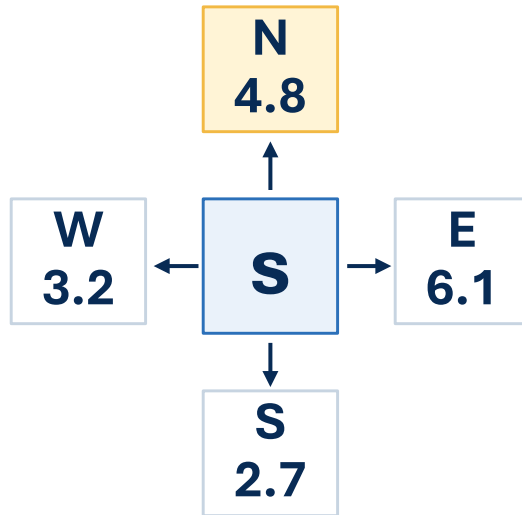
Consider the available actions and choose the largest estimated long-term value.

$$\pi_{\text{new}}(s) = \arg \max_a Q^\pi(s, a)$$

Improvement chooses the greedy action according to the evaluated values.

Improvement chooses the greedy action

Find the highest action value.



current: N

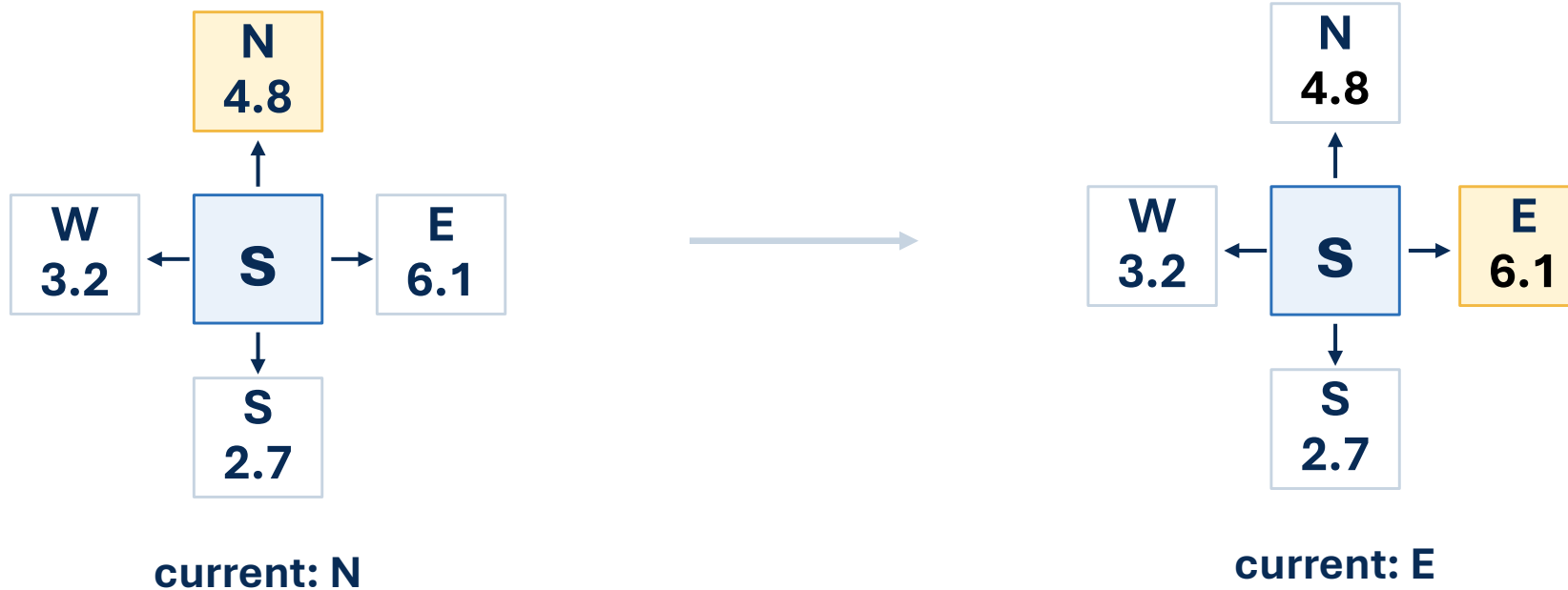
Largest Q-value
East has the largest action value: 6.1

Improved action
 $\pi_{\text{new}}(s) = \text{East}$

We are no longer evaluating the old arrow; we are choosing the best arrow.

Sometimes the arrow changes

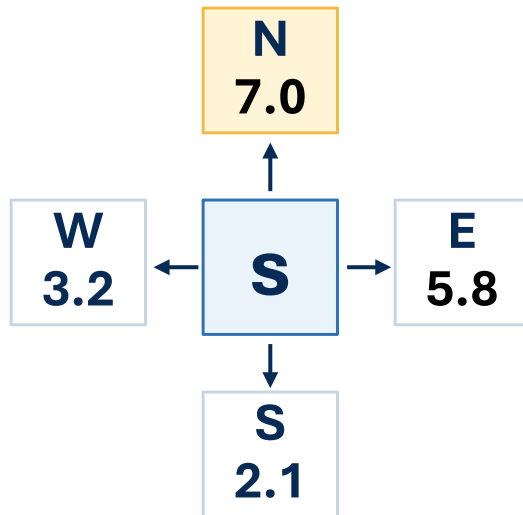
Improvement can replace a current action.



North → East because East gives the largest estimated long-term return.

Sometimes the arrow stays exactly where it is

Policy improvement does not require a change.



current: N

Greedy action

North is already the largest value.

Conclusion

Improvement happened, but this state's policy entry stays North.

Rule: choose the greedy action — not “always change the action.”

The memory cue

Evaluation changes values. Improvement changes actions.

→ -1.9	→ 9.6	→ 16.1	+10
↓ -1.9	WALL	↓ -3.8	-10
→ -1.9	→ -1.9	→ -12.4	↑ -16.9



→ 5.6	→ 12.3	→ 16.1	+10
↑ -2.7	WALL	↑ -12.1	-10
↑ -2.7	← -10.3	← -14.6	↓ -18.0

LOCKED policy

Arrows stay fixed.
Cell values change.

UNLOCKED policy

Values guide decisions.
Arrows may change.

LOCK arrows → update numbers. UNLOCK arrows → choose better actions.

One complete policy-iteration cycle

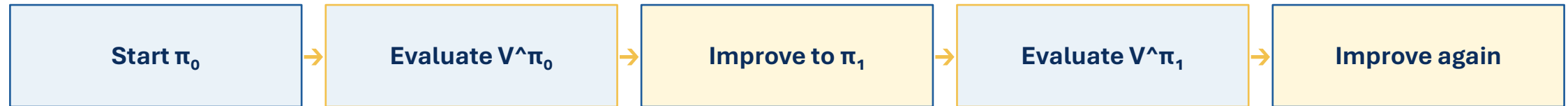
A policy becomes a value estimate, then an improved policy.



Policy iteration alternates: evaluate the current policy, then improve it.

Then do it again

The improved policy becomes the new current policy.



State	π_0	π_1
A	Charge	Maintain
B	Maintain	Maintain
C	Shed	Shed

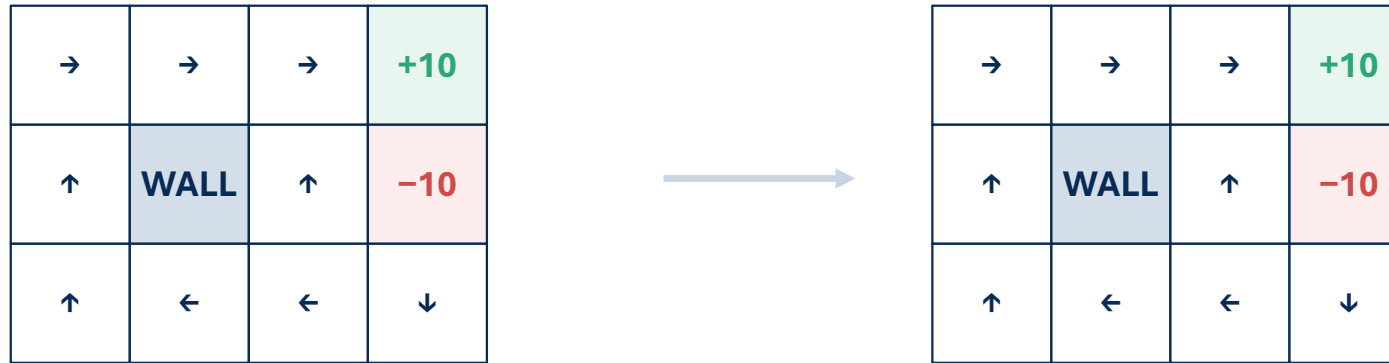
Decision point

Because at least one action changed, the new policy must be evaluated next.

The loop continues until improvement produces no changes.

When do we stop?

Policy iteration terminates when the policy is stable.



No arrows changed → policy stable → stop.

Policy stability is different from value convergence

Different algorithms use different stopping signals.

Value Iteration

Stop when value estimates change by less than a tolerance.

Δ is small.

Policy Iteration

Stop when improvement produces the same policy.

No arrows changed.

VI watches the numbers. PI watches the arrows.

VI vs. PI: same ingredients, different organization

Both use model-based Bellman reasoning.

Value Iteration

$V_0 \rightarrow$ Bellman optimality backup $\rightarrow V_1$

MAX occurs inside each backup.

Policy Iteration

$\pi_0 \rightarrow$ evaluate $\pi_0 \rightarrow$ improve $\rightarrow \pi_1$

MAX occurs during improvement.

VI mixes judging and choosing. PI separates judging from choosing.

Where does MAX happen?

The placement of MAX tells the story.

Value Iteration

For every backup:

possible actions $\rightarrow$ MAX $\rightarrow$ updated V

Policy Iteration

Evaluation:
follow π only

Improvement:
compare actions $\rightarrow$ MAX $\rightarrow$ new π

No MAX during evaluation of a fixed deterministic policy.

Both are model-based

Policy iteration still needs the MDP model.

States

Actions

**Transition
probabilities**

Rewards

Needed for evaluation
 $P(s'|s,a)$ and R determine the expected return under the policy.

Next question
What if those transition probabilities are unknown?

This sets up Q-learning: learning from experience instead of a known model.

Practice: policy-improvement sprint

Use evaluated action values to improve the policy.

State	Current action	Q(Hold)	Q(Charge)	Improved action
P	Hold	4	6	_____
Q	Charge	7	5	_____
R	Hold	3	2	_____

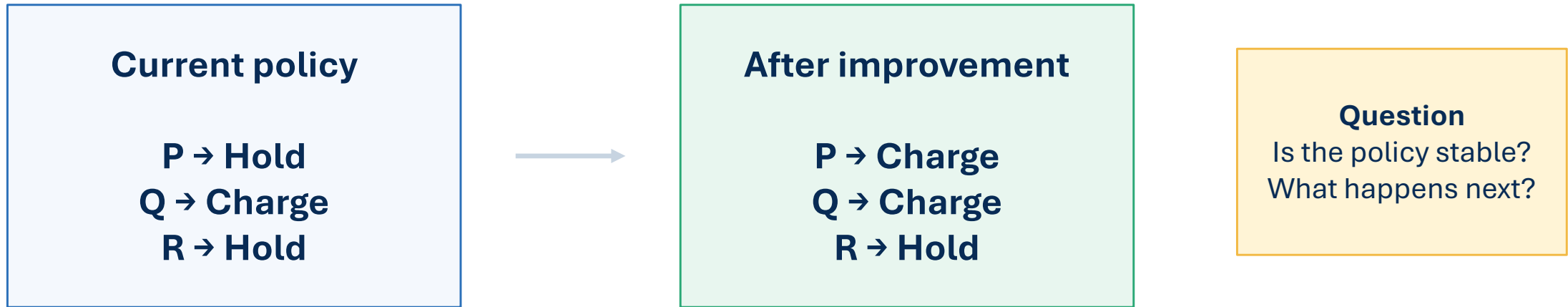
Tasks

1. Fill in the improved action.
2. Identify which states changed.
3. Decide whether the policy is stable.

Improvement chooses the highest action value at each state.

Mini cycle: are we done?

Compare the current policy to the improved policy.



If any action changes, the new policy must be evaluated.

Practice: diagnose the mistake

Label each statement as correct or incorrect.

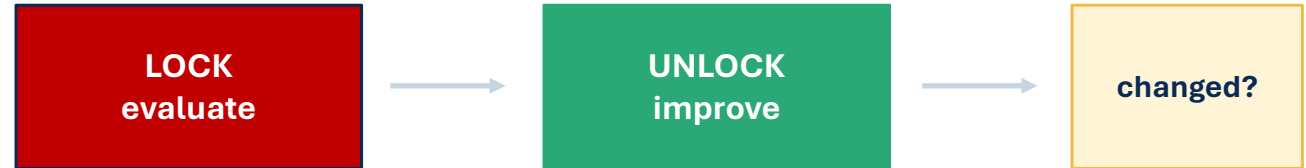
Statement	Answer
Policy evaluation finds the optimal policy.	Incorrect
Evaluation of a fixed deterministic policy uses MAX over actions.	Incorrect
Improvement must always change the action.	Incorrect
Policy iteration stops when no policy entries change.	Correct

The most common errors confuse evaluation with improvement.

Read policy iteration

Map each part of the pseudocode to a visual phase.

```
choose initial policy  $\pi$ 
repeat:
  evaluate  $\pi$ 
  stable = true
  for each state  $s$ :
    old =  $\pi(s)$ 
     $\pi(s)$  = best action using  $V^\pi$ 
    if  $\pi(s) \neq \text{old}$ :
      stable = false
until stable
```



Questions

Which lines evaluate?
Which lines improve?
Why save old?
What causes termination?

The loop ends only when the improvement pass makes no changes.

Exit ticket: rebuild policy iteration

Complete the two-phase story.

Policy evaluation

Keep _____ fixed and estimate _____.

Policy improvement

Choose the _____ action using the evaluated values.

If at least one action changes
_____.

If no actions change
_____.

One-sentence comparison

How does policy iteration differ from value iteration?

Next: What if we do not know the transition probabilities at all?